

向量数据库入门指南

什么是向量数据库

向量数据库是专门用于存储和检索高维向量的数据库系统。传统数据库存储结构化数据，通过精确匹配查询；向量数据库存储的是嵌入向量（embedding），通过计算相似度来检索。

这种特性使它非常适合处理"语义搜索"场景：不需要关键词完全匹配，只需要意思相近，就能找到相关内容。比如搜索"如何提升服务器性能"，也能匹配到"服务器优化方案"或"减少系统响应延迟"这类文档。

向量的生成原理

文本、图片、音频等非结构化数据，无法直接计算相似度。向量化的过程就是把这些数据转换成一组数字——一个固定长度的浮点数数组。这个数组在高维空间中表示一个点，语义相近的内容在这个空间里会聚集在一起。

以文本为例，一句话经过嵌入模型处理后，得到一个 1536 维的向量（OpenAI text-embedding-3-small 的输出维度）。这个向量捕捉了句子的语义信息：同一话题的句子，对应的向量方向相近，余弦相似度高。

生成向量的模型叫做嵌入模型（embedding model）。常见的选择有 OpenAI 的 text-embedding-3-small、Cohere 的 embed-v3，以及专门为中文优化的 BGE 系列模型。不同模型产生的向量维度不同，不能混用——用 A 模型建的索引，必须用同一个 A 模型来查询。

主流向量数据库对比

目前市场上的向量数据库分为两类：专用向量数据库和带向量扩展的传统数据库。

专用向量数据库包括 Qdrant、Weaviate、Milvus、Pinecone 等。它们从底层为向量检索设计，在性能、扩展性、过滤能力上都有优势。其中 Qdrant 用 Rust 编写，内存效率和查询速度表现突出，社区活跃，是目前工程实践中的热门选择。

传统数据库的向量扩展包括 pgvector（PostgreSQL 插件）和 Elasticsearch 的 kNN 功能。如果团队已经有 PostgreSQL 基础设施，pgvector 是低成本的接入方式，但在百万级以上数据量时检索性能会下降，需要建立 HNSW 索引来加速。

选型的核心原则：数据量在百万条以内、团队已有 PostgreSQL 的，pgvector 够用；数据量更大、需要复杂 payload 过滤的，建议直接上 Qdrant。

相似度计算方式

向量数据库支持三种常见的相似度计算方式。

余弦相似度（cosine similarity）衡量两个向量方向的夹角。方向越接近，相似度越高，值域在 -1 到 1 之间。文本语义检索通常用余弦相似度，因为它不受向量长度影响，只关注方向。

点积（dot product）是两个向量对应维度相乘后求和。计算速度比余弦快，但受向量模长影响，适合向量已经做过归一化处理的场景。

欧氏距离（L2 distance）衡量两个点在空间中的直线距离。距离越小，越相近。适合图像检索等场景，文本语义检索较少使用。

大多数嵌入模型的输出都是归一化的向量，这种情况下余弦相似度和点积计算结果等价，可以选速度更快的点积。

RAG 中的向量检索流程

在 RAG 系统里，向量数据库承担的角色是"记忆库"。整个流程分为写入和检索两个阶段。

写入阶段：把文档切成适当大小的 chunk，用嵌入模型把每个 chunk 转成向量，连同 chunk 原文和元数据一起写进向量数据库。这个阶段通常只跑一次，或者在文档更新时增量更新。

检索阶段：用户提问时，先把问题也转成向量（必须用同一个嵌入模型），然后在向量数据库里找出相似度最高的 top-K 个 chunk，把这些 chunk 的原文作为上下文拼进 prompt，送给 LLM 生成回答。

影响检索质量的关键变量：嵌入模型的选择、chunk 大小、top-K 数量，以及是否使用混合检索（向量检索 + 关键词检索结合）。这些参数没有通用最优值，需要针对具体数据集做评测。

实际部署注意事项

向量数据库在生产环境中有几个常见的踩坑点。

索引更新延迟。Qdrant、Weaviate 等数据库在写入向量后，索引不是实时生效的，有几百毫秒到几秒的延迟。实时性要求高的场景需要注意这一点，避免写入之后立刻查询拿不到最新数据。

内存占用。向量索引通常要全量加载到内存才能高速查询。一百万条 1536 维的 float32 向量，光原始数据就约 6GB，HNSW 索引还会再增加 20%-50% 的内存开销。评估资源时要预留足够的 RAM。

批量写入性能。单条写入的 API 调用开销是固定的，大量文档入库时应当使用批量写入接口 (batch upsert)，把几百条 chunk 打包成一次请求，吞吐量可以提升几十倍。